

# 02\_Run\_Extraction: Straight-Line Coastdown Interval Detection

Validation\_Aero\_Test

August 4, 2026

## 1 Overview

This document describes the straight-line interval detection used to extract coastdown runs from the cleaned telemetry dataset. The implementation lives in `aero_validation/segmentation.py` and is invoked via the CLI script `Code/02_Run_Extraction/export_drag_runs.py`.

### Pipeline flow

```
cleaned_10ms.csv (from 01_Data_Preparation)
  |
  v
export_drag_runs.py --> drag_intervals.csv + overview plot
  |
  v
generate_run_outputs.py --> run_XXX.csv + per-run QC plots
  |
  v
downstream (drag calculation, 03_Drag_Calculation)
```

The algorithm:

- proposes candidate run starts using speed peak detection,
- extends forward until a stopping condition (turning/braking/re-acceleration/stop),
- enforces quality gates (straightness, duration),
- writes an intervals CSV used by downstream steps.

**Provenance.** The example outputs shown in this document are from the Steißlingen aero-validation test day (2025-10-22), comprising 4 stints and 46 extracted coastdown runs.

## 2 Inputs

From the cleaned dataset CSV produced by the data-preparation pipeline (see `01_Data_Preparation.pdf`):

- **Time:** `t_s` [s]
- **Speed:** `GPS1_VEL_MAG_ms` [ $\text{m s}^{-1}$ ]
- **Steering:** `ICU_SteeringAngle_deg` [deg]
- **Optional brakes:** `ICU_Brakpresf_bar`, `ICU_Brakpresr_bar` [bar]

### 3 Reproducibility

The following commands reproduce the run extraction from the cleaned dataset (run from the repository root):

```
# Detect coastdown intervals and write the intervals CSV
python Code/02_Run_Extraction/export_drag_runs.py \
    --csv Outputs/01_Data_Preparation/Aero_Validation_Signals_cleaned_10ms.csv \
    --out Outputs/02_Run_Extraction/csv/drag_intervals.csv

# Generate per-run time-series slices and QC plots
python Code/02_Run_Extraction/generate_run_outputs.py \
    --csv Outputs/01_Data_Preparation/Aero_Validation_Signals_cleaned_10ms.csv \
    --intervals Outputs/02_Run_Extraction/csv/drag_intervals.csv \
    --outdir Outputs/02_Run_Extraction/csv
```

All thresholds can be overridden via CLI flags; see Section 10 for the canonical defaults.

### 4 Preprocessing

Let  $t$  be the sample times and  $v$  the speed signal.

- Estimate the median sample period  $\Delta t_{\text{med}}$  from  $\Delta t = \text{diff}(t)$ .
- Convert time windows (seconds) to sample windows by  $n \approx \text{round}(\Delta t_{\text{window}} / \Delta t_{\text{med}})$ .
- If smoothing is enabled (default): smooth  $v$  with a centered rolling mean using a window derived from `peak_window_s`.
- If brake signals exist: combine  $b(t) = \max(b_f, b_r)$  and lightly smooth with a centered rolling mean of 5 samples.

### 5 Candidate peak detection

Peak detection is implemented in `detect_drag_peaks(t, v, config)`. It combines two mechanisms and returns sorted unique indices:

1. **Local maxima:** indices where  $v_s[i] \geq v_s[i-1]$  and  $v_s[i] \geq v_s[i+1]$ .
2. **Derivative sign change:** indices where the speed derivative changes from clearly positive to non-positive.

**Prominence gate** For each local maximum, compute a prominence proxy using the 60th percentile of the speed within a local window around the candidate:

$$p = v_s[i] - P_{60}(v_s \text{ in a local window around } i),$$

and require  $v_s[i] \geq \text{min\_peak\_speed}$  and  $p \geq \text{peak\_prominence}$ .

## 6 Forward extension and end conditions

Given a start index  $p$  (peak), the algorithm extends forward until the first end condition triggers. Let  $i$  be the current index and  $j = i + \text{accel\_window\_s}$  (converted to samples).

End conditions (evaluated sequentially):

- **Stop:**  $v_s[i] \leq \text{stop\_speed\_th}$ .
- **Turning:**  $|\theta[i]| > \text{steer\_thresh}$  sustained for  $\text{turn\_window\_s}$ .
- **Braking:**  $b[i] > \text{brake\_thresh}$  or an instantaneous brake “spike” exceeds  $\text{brake\_spike\_thresh}$ .
- **Re-acceleration:** if  $v_s[j] - v_s[i] > \text{reaccel\_eps}$ .

The returned interval is half-open:  $(p, e)$  where  $e$  is the first index after the accepted end.

## 7 Quality gates

After computing  $(p, e)$ :

- **Straightness:** the fraction of samples in  $[p, e)$  where  $|\theta| \leq \text{steer\_thresh}$  must be at least  $\text{straight\_ratio}$ .
- **Duration:** the run duration  $t[e-1] - t[p]$  must satisfy  $\text{min\_dur\_s} \leq \text{duration} \leq \text{max\_dur\_s}$ .
- **Non-overlap:** candidate peaks that would overlap a previously accepted interval are skipped.

## 8 Error handling & robustness

- **No peaks found:** if no speed peaks satisfy the prominence and minimum-speed gates, an empty intervals CSV is produced (no crash).
- **Missing steering:** if `ICU_SteeringAngle_deg` is absent, the straightness gate is skipped and all runs are accepted on duration alone. A warning should be raised in this case.
- **Missing brakes:** if brake pressure channels are absent, the braking end-condition is simply not evaluated; runs terminate only on stop, turn, or re-acceleration.
- **Short dataset:** if fewer than 3 samples exist, segmentation returns an empty list immediately.

## 9 CLI usage and outputs

### Export intervals

`Code/02_Run_Extraction/export_drag_runs.py` loads the cleaned dataset, runs segmentation, writes an intervals CSV, and generates an overview plot.

Default output paths:

- Intervals CSV: `Outputs/02_Run_Extraction/csv/drag_intervals.csv`
- Overview plot: `Outputs/02_Run_Extraction/plots/velocity_overview_with_runs.png`

Intervals CSV schema (key fields):

- `run_id`: 1..N

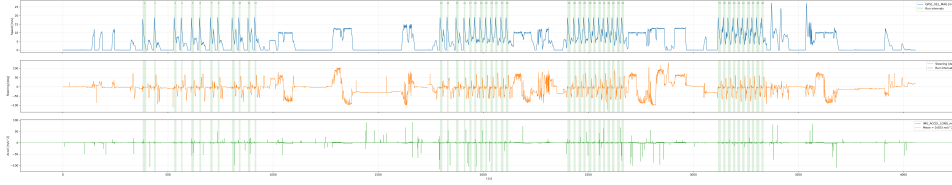


Figure 1: Overview of detected coastdown intervals overlaid on the speed trace (generated by `export_drag_runs.py`).

- `start_idx`, `end_idx`: index range into the input dataset (end exclusive)
- `t0_s`, `t1_s`, `duration_s`
- `v_start_ms`, `v_end_ms`
- `source_basename`, `row_count`: metadata for mismatch detection

## Optional debug log

If `-debug-out` is provided, `segment_drag_runs_with_debug` writes a per-peak decision log with the following columns:

|                                   |                                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>peak_idx</code>             | Index of the candidate peak                                                                                                         |
| <code>t_peak</code>               | Time at the peak [s]                                                                                                                |
| <code>accepted</code>             | Boolean: did this peak yield a valid run?                                                                                           |
| <code>reason</code>               | Rejection reason if not accepted ( <code>overlap</code> , <code>duration</code> , <code>straight</code> )                           |
| <code>start_idx</code>            | Start index of the accepted interval                                                                                                |
| <code>end_idx</code>              | End index (exclusive)                                                                                                               |
| <code>t0</code> , <code>t1</code> | Start and end times [s]                                                                                                             |
| <code>duration</code>             | Run duration [s]                                                                                                                    |
| <code>steer_ratio</code>          | Fraction of samples within steering threshold                                                                                       |
| <code>end_reason</code>           | What ended the run ( <code>stop</code> , <code>turn</code> , <code>brake</code> , <code>brake_spike</code> , <code>reaccel</code> ) |

## 10 Default thresholds

The canonical defaults are defined in the `DragRunConfig` dataclass (`aero_validation/segmentation.py`) and can be overridden individually via CLI flags or programmatically by passing a custom instance.

The defaults exposed by `export_drag_runs.py` are:

|                                 |           |
|---------------------------------|-----------|
| <code>steer_thresh</code>       | 10 deg    |
| <code>straight_ratio</code>     | 0.75      |
| <code>min_peak_speed</code>     | 15.0 m/s  |
| <code>peak_window_s</code>      | 0.5 s     |
| <code>peak_prominence</code>    | 0.05 m/s  |
| <code>reaccel_eps</code>        | 0.2 m/s   |
| <code>accel_window_s</code>     | 0.5 s     |
| <code>brake_thresh</code>       | 0.05 bar  |
| <code>brake_spike_thresh</code> | 0.1 bar/s |
| <code>turn_window_s</code>      | 0.3 s     |
| <code>min_dur_s</code>          | 1.5 s     |
| <code>max_dur_s</code>          | 20.0 s    |
| <code>stop_speed_th</code>      | 0.2 m/s   |

## 11 Per-run outputs

The script `Code/02_Run_Extraction/generate_run_outputs.py` consumes `drag_intervals.csv` and writes:

- `Outputs/02_Run_Extraction/csv/run_{XXX}.csv`: sliced time series per interval
- `Outputs/02_Run_Extraction/plots/run_{XXX}_velocity_steering_brake.png`: QC plots per run

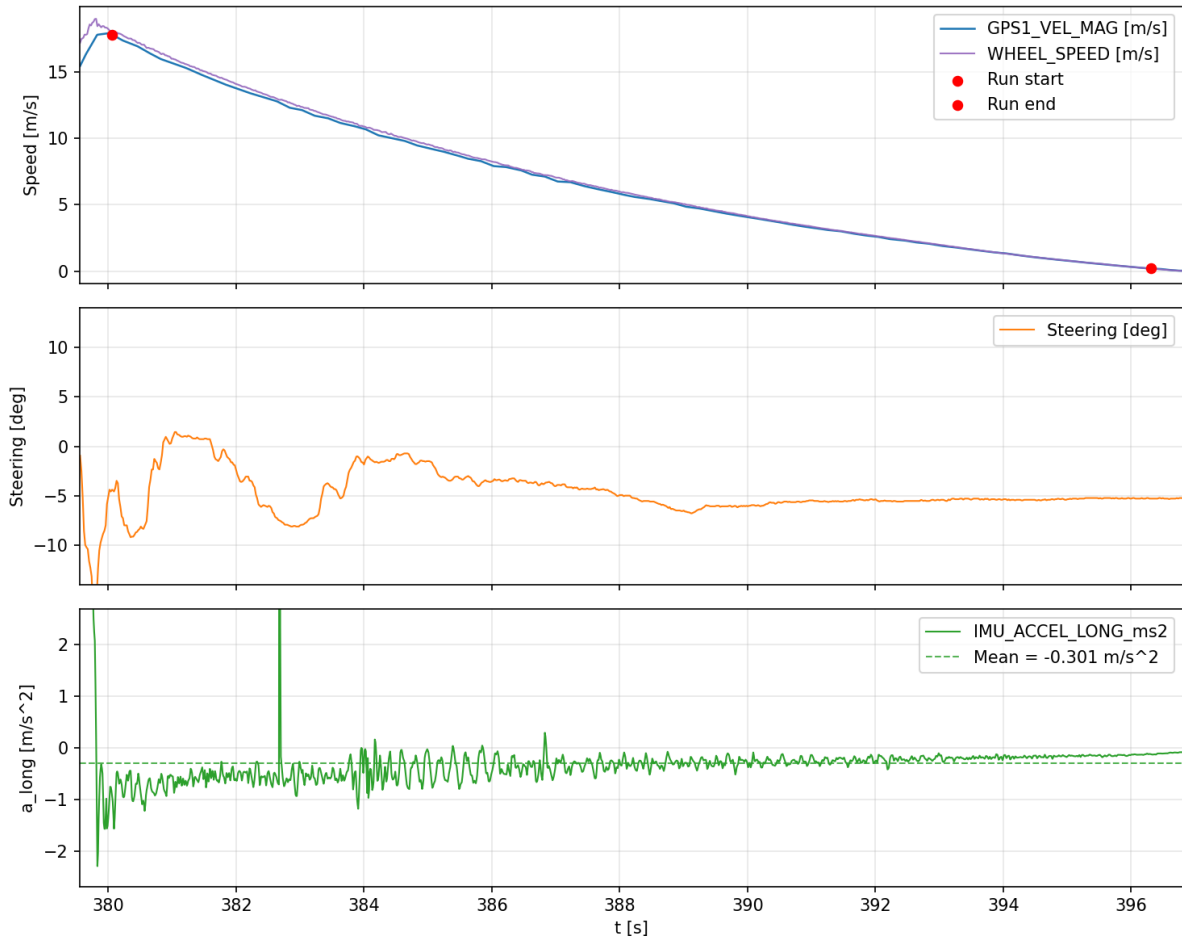


Figure 2: Example per-run QC plot (speed, steering, and brake traces) for Run 001.

The intervals CSV produced here is the input for the drag coefficient estimation step (see `03_Drag_Calculation.pdf`).